

## Glosario - Semana 07

### React Router v6 con TypeScript

---

#### A

##### Active Route (Ruta Activa)

La ruta que actualmente coincide con la URL del navegador. En React Router, `NavLink` puede detectar si es la ruta activa para aplicar estilos diferentes.

```
// NavLink aplica 'active' automáticamente a la ruta activa
<NavLink to="/about" className={({ isActive }) => isActive ? 'active' : ''}>
  About
</NavLink>
```

---

#### B

##### BrowserRouter

Componente que utiliza la History API del navegador para mantener la UI sincronizada con la URL. Es el router más común para aplicaciones web.

```
import { BrowserRouter } from 'react-router-dom';

<BrowserRouter>
  <App />
</BrowserRouter>
```

##### Back Navigation

Navegación hacia atrás en el historial del navegador, que se puede implementar programáticamente.

```
const navigate = useNavigate();
navigate(-1); // Ir hacia atrás
navigate(-2); // Ir 2 páginas atrás
```

---

#### C

##### Client-Side Routing (Enrutamiento del Lado del Cliente)

Técnica donde la navegación entre páginas es manejada por JavaScript en el navegador, sin hacer peticiones al servidor para cada cambio de página.

##### Catch-All Route (Ruta Comodín)

Ruta que captura todas las URLs que no coinciden con ninguna otra ruta definida. Se usa para páginas 404.

```
<Route path="*" element={<NotFoundPage />} />
```

---

## D

### Dynamic Route (Ruta Dinámica)

Ruta que contiene segmentos variables, representados con `:` seguido de un nombre de parámetro.

```
// :bookId es un segmento dinámico
<Route path="/books/:bookId" element={<BookDetail />} />
```

### Deep Linking

Capacidad de enlazar directamente a una página específica dentro de una aplicación, incluyendo parámetros de URL y query strings.

---

## E

### element prop

Propiedad de `<Route>` que especifica el componente React a renderizar cuando la ruta coincide.

```
<Route path="/about" element={<AboutPage />} />
```

### end prop

Propiedad de `NavLink` que indica que solo debe marcarse como activo si la ruta coincide exactamente.

```
<NavLink to="/" end>Home</NavLink>
```

---

## G

### Guard (Guardia de Ruta)

Componente o lógica que protege rutas verificando condiciones antes de permitir el acceso.

```
const ProtectedRoute = ({ isAuthenticated, children }) => {
  if (!isAuthenticated) return <Navigate to="/login" />;
  return children;
};
```

---

## H

### HashRouter

Router que usa el hash (#) de la URL para simular diferentes rutas. Útil cuando no se puede configurar el servidor para manejar rutas.

```
import { HashRouter } from 'react-router-dom';
// URLs lucen como: example.com/#/about
```

### History API

API del navegador que permite manipular el historial de navegación sin recargar la página. Es la base de `BrowserRouter`.

---

## I

### Index Route (Ruta Índice)

Ruta hija que se renderiza cuando la URL coincide exactamente con la ruta padre.

```
<Route path="dashboard" element={<DashboardLayout />}>
  <Route index element={<DashboardHome />} /> {/* Se muestra en /dashboard */}
  <Route path="settings" element={<Settings />} />
</Route>
```

#### isActive

Propiedad booleana disponible en el callback de `className` o `style` de `NavLink` que indica si la ruta está activa.

---

## L

### Layout Route (Ruta de Layout)

Ruta que define un layout compartido para sus rutas hijas, usando `<Outlet>` para renderizar el contenido dinámico.

#### <Link>

Componente para navegación declarativa que previene la recarga de página.

```
<Link to="/about">About</Link>
<Link to="/books/123">Book Detail</Link>
```

### Loader

Función en React Router v6.4+ que carga datos antes de renderizar una ruta.

---

## M

### MemoryRouter

Router que mantiene el historial en memoria, sin modificar la URL del navegador. Útil para testing.

### MPA (Multi-Page Application)

Aplicación tradicional donde cada navegación resulta en una petición al servidor y recarga completa de la página.

---

## N

#### <Navigate>

Componente que redirige automáticamente a otra ubicación cuando se renderiza.

```
if (!isAuthenticated) {
  return <Navigate to="/login" replace />;
}
```

#### <NavLink>

Versión especial de `<Link>` que sabe si está "activa" basándose en la URL actual.

## Nested Routes (Rutas Anidadas)

Rutas definidas dentro de otras rutas, creando jerarquías de componentes.

```
<Route path="dashboard" element={<Dashboard />}>
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>
```

## O

### <Outlet>

Componente que renderiza el contenido de la ruta hija actual dentro de un layout padre.

```
const Layout = () => (
  <div>
    <Header />
    <Outlet /> { /* Aquí se renderiza la ruta hija */ }
    <Footer />
  </div>
);
```

## P

### Path Parameters (Parámetros de Ruta)

Segmentos dinámicos de una URL que se extraen usando `useParams`.

```
// URL: /books/123
const { bookId } = useParams<{ bookId: string }>();
// bookId = "123"
```

### Protected Route (Ruta Protegida)

Ruta que requiere autenticación u otra condición para ser accesible.

### Programmatic Navigation

Navegación ejecutada mediante código JavaScript, no por clicks del usuario.

```
const navigate = useNavigate();
navigate('/dashboard');
```

## Q

### Query String (Cadena de Consulta)

Parte de la URL después del `?` que contiene pares clave-valor para pasar datos.

```
// URL: /books?genre=fiction&sort=title
const [searchParams] = useSearchParams();
const genre = searchParams.get('genre'); // "fiction"
```

---

## R

### replace

Opción de navegación que reemplaza la entrada actual del historial en lugar de agregar una nueva.

```
navigate('/home', { replace: true });
<Navigate to="/login" replace />
```

### <Route>

Componente que define una relación entre una URL y el componente a renderizar.

### <Routes>

Contenedor que agrupa múltiples <Route> y selecciona el que mejor coincide con la URL actual.

---

## S

### SPA (Single-Page Application)

Aplicación que carga una sola página HTML y actualiza dinámicamente el contenido sin recargar.

### state

Datos adicionales que se pueden pasar durante la navegación, accesibles en la página de destino.

```
// Enviar state
navigate('/dashboard', { state: { from: '/login' } });

// Recibir state
const location = useLocation();
const { from } = location.state || {};
```

---

## U

### useLocation

Hook que retorna el objeto location actual, incluyendo pathname, search, hash y state.

```
const location = useLocation();
console.log(location.pathname); // "/books"
console.log(location.search); // "?genre=fiction"
```

### useNavigate

Hook que retorna una función para navegación programática.

```
const navigate = useNavigate();
navigate('/about');
navigate(-1); // Atrás
```

### useParams

Hook que retorna un objeto con los parámetros de ruta de la URL actual.

```
// Ruta: /books/:bookId
const { bookId } = useParams<{ bookId: string }>();
```

### useSearchParams

Hook que permite leer y modificar los query parameters de la URL.

```
const [searchParams, setSearchParams] = useSearchParams();
const genre = searchParams.get('genre');
setSearchParams({ genre: 'fiction' });
```

## URL Parameters

Ver "Path Parameters" y "Query String".

---

## W

### Wildcard Route

Ver "Catch-All Route".

---

## Símbolos

#### : (Dos Puntos)

Prefijo que indica un segmento dinámico en una ruta.

```
<Route path="/users/:userId" element={<UserProfile />} />
```

#### \* (Asterisco)

Coincide con cualquier ruta. Se usa para páginas 404.

```
<Route path="*" element={<NotFound />} />
```

#### ? (Interrogación)

En URLs, inicia la cadena de query parameters.

```
/search?query=react&page=1
```